

Interarrival Time (IAT) Calculator

Product Requirements Document — v2

Portfolio Project · Built with Google AI Studio + Make.com

1. What This Tool Does (Plain Summary)

Companies that ship physical products need to keep some inventory on hand as a buffer between deliveries. How much buffer they need depends on how often deliveries actually arrive - not how often they're supposed to arrive.

This tool measures how often a product actually arrived at a warehouse, using real delivery records instead of a planned delivery schedule. It then shows what that real number means in practical terms - specifically, how much inventory buffer is actually needed, and whether the business is currently over- or under-stocked as a result.

It also includes an AI assistant that can explain any result in plain language and answer "what if" questions, and a scheduled automation that refreshes the whole analysis every week without anyone running it manually.

2. The Business Problem

Warehouses typically plan their inventory buffer around a “planned lead time” - an assumption of how long delivery takes. In practice, real deliveries don't always match that assumption: trucks get consolidated, some warehouses only receive on certain days, and delivery patterns drift over time without anyone updating the planning assumption.

When the assumption is wrong, the consequence is real money: too little buffer risks running out of stock, and too much buffer ties up cash and warehouse space that didn't need to be spent. This tool replaces the assumption with a number calculated directly from real delivery history, then quantifies the dollar impact of the gap between the two.

3. Who Would Use Something Like This

- A supply planner deciding how much buffer stock a warehouse should carry
- An inventory analyst checking which numbers are based on real data versus an estimate
- A team launching a new product that has no delivery history yet, and needs a starting estimate

4. What Was Built (v1 Scope)

- A calculator that works out the real delivery frequency for each product at each warehouse, from historical delivery records
- A fallback system for brand-new products with no delivery history yet - it borrows a reasonable estimate from a closely related product, and only from products with real data of their own
- A dashboard to browse, filter, and search all results
- A “Business Impact” view that translates the delivery-frequency number into an actual inventory and dollar impact
- An AI assistant that explains any result and can answer hypothetical questions
- A weekly automation that refreshes everything and emails a summary, with no manual work required

Not included in v1

- A live connection to a real company system - this version runs on uploaded/sample data
- Automatically updating a planning system - results are informational, not a direct write-back

5. How the Core Calculation Works

Every warehouse is one of two types, and each type is measured differently, because a large regional “Hub” warehouse and a smaller local “Spoke” warehouse naturally receive deliveries on different rhythms:

Warehouse Type	How It's Measured	Why
Hub DC	Count how many separate weeks (out of the last 26) had at least one delivery, then divide 182 days by that count	Hub warehouses typically receive larger, less frequent shipments - a weekly view fits that pattern
Spoke DC	Count how many separate days (out of the last 182) had at least one delivery, then divide 182 days by that count	Spoke warehouses typically receive smaller, more frequent shipments - a daily view fits that pattern

For a brand-new product with no delivery history yet at a given warehouse, the tool tries three things in order, and stops at the first one that works:

- 1. Look for another product with the exact same barcode family that already has a real, calculated number at that same warehouse - use that.
- 2. If none, look for another product in the same category that has a real, calculated number - use the average of those.
- 3. If nothing is found either way, assume the maximum: 182 days (i.e., treat it as an unknown until real data accumulates).

Important rule: this fallback only ever borrows from a product with real, directly-measured data - never from another product that was itself estimated. That prevents small errors from stacking up the further a product gets from real delivery history.

6. Business Impact (Cycle Stock)

Once a product's real delivery frequency is known, the tool compares two versions of the inventory buffer it implies:

Version	Formula	Meaning
Actual buffer needed	Daily demand $\times$ real delivery frequency $\div$ 2	Based on how the warehouse actually receives this product
Naive buffer assumed	Daily demand $\times$ planner's assumed lead time $\div$ 2	Based on the planning assumption that may be outdated

The difference between these two numbers, multiplied by unit cost, is the dollar impact - and its direction tells a clear story:

- If the real buffer needed is higher than assumed $\rightarrow$ the business is understocked and carrying stockout risk
- If the real buffer needed is lower than assumed $\rightarrow$ the business is holding excess stock and tying up cash unnecessarily
- If they match $\rightarrow$ no action needed

7. AI Assistant

A chat panel lets anyone ask questions in plain English and get an answer grounded in the tool's actual calculated data - never a guess. It can do three things:

Ask it to...	What it does
Explain a specific result ("Why is this product's number 26 days?")	Looks up the exact calculation behind that result and explains it in plain language

Ask it to...	What it does
Answer a ranking/comparison question (“Which products have the longest delivery gap?”)	Sorts and filters the real results table, then summarizes it - it does not do this math itself
Answer a what-if question (“What if we checked this as of June instead?”)	Re-runs the exact same calculation with a different date and shows what would change, without altering the live dashboard

Design principle: the AI never performs the underlying math itself. It only reads back numbers the calculator already produced. This avoids the AI ever inventing or misstating a figure.

8. Automation (Weekly Refresh)

A separate no-code automation (built in Make.com) runs the entire calculation on a weekly schedule without anyone opening the dashboard:

- Pulls the latest source data from a connected spreadsheet
- Re-runs the exact same calculation logic as the dashboard (not a separate copy of it)
- Writes the updated results back to a results spreadsheet
- Emails a short summary: how many products were recalculated, how many are now flagged for stockout or excess-stock risk, and the total dollar impact

This turns the tool from something that has to be run manually into something that runs itself and only asks for attention when something needs it.

9. What Was Learned Building This

This project was built by directing an AI app-building tool with detailed prompts, then testing the output like a real deliverable - not assuming it was correct on the first try. Several real defects were found and fixed along the way, including two that would have silently given the wrong business conclusion if they'd gone unnoticed. Full details are in the companion UAT Test Cases & Results document and the AI Studio Prompts Log.

- Two calculation engines (the dashboard's and a later simulation feature) must share one function, or they will quietly drift apart - this happened once and caused a real defect.
- A fallback/estimation rule needs a rule against estimating from another estimate, or small errors compound invisibly.
- Testing against a hand-built answer key - not just checking that the output “looks reasonable” - is what actually catches these issues.